JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util.function

# Interface IntPredicate

**Functional Interface:**

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

---

```
@FunctionalInterface
public interface IntPredicate
```

Represents a predicate (boolean-valued function) of one `int`-valued argument. This is the `int`-consuming primitive type specialization of `Predicate`.

This is a functional interface whose functional method is `test(int)`.

**Since:**

1.8

**See Also:**

`Predicate`

## *Method Summary*

Abstract Methods | Default Methods

| Modifier and Type | Method | Description |
|---|---|---|
| default `IntPredicate` | `and(IntPredicate other)` | Returns a composed predicate that represents a short-circuiting logical AND |

| | | |
|---|---|---|
| | | of this predicate and another. |
| default IntPredicate | negate() | Returns a predicate that represents the logical negation of this predicate. |
| default IntPredicate | or(IntPredicate other) | Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. |
| boolean | test(int value) | Evaluates this predicate on the given argument. |

## *Method Detail*

### test

```
boolean test(int value)
```

Evaluates this predicate on the given argument.

**Parameters:**

`value - the input argument`

**Returns:**

`true if the input argument matches the predicate, otherwise false`

### and

```
default IntPredicate and(IntPredicate other)
```

Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. When evaluating the composed predicate, if this predicate is `false`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

**Parameters:**

other - a predicate that will be logically-ANDed with this predicate

**Returns:**

a composed predicate that represents the short-circuiting logical AND of this predicate and the other predicate

**Throws:**

NullPointerException - if other is null

### negate

```
default IntPredicate negate()
```

Returns a predicate that represents the logical negation of this predicate.

**Returns:**

a predicate that represents the logical negation of this predicate

### or

```
default IntPredicate or(IntPredicate other)
```

Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. When evaluating the composed predicate, if this predicate is `true`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

**Parameters:**

other - a predicate that will be logically-ORed with this predicate

**Returns:**

a composed predicate that represents the short-circuiting logical OR of this predicate and the other predicate

**Throws:**

```
NullPointerException - if other is null
```



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.